



UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA

MI PRIMER DASHBOARD ENTREGABLE 2

ESTUDIANTE:
ANDY WILFRIDO QUIMI QUIMI

MAESTRO:
ING. ANTHONY PACHAY ESPINOZA, MSC

MATERIA:
INTELIGENCIA DE NEGOCIOS

CURSO:

5

PARALELO:

1

AÑO:
2026-2027

Fase 2 – Modelo dimensional en PostgreSQL

Guía teórica y documentación del entregable

Proyecto BI – Dataset Brazilian E-Commerce (Olist). Materia: Calidad de Software, UPSE.

Contenido

- 1. Mapa de la fase
- 2. Teoría esencial (para entender lo que haces)
- 3. Modelo dimensional definitivo
- 4. Transformaciones realizadas sobre el dataset
- 5. Consultas de validación
- 6. Vistas y funciones SQL
- 7. Mockup del dashboard
- 8. Matriz de trazabilidad
- 9. Evidencias que debes capturar
- 10. Preguntas para defender el trabajo
- 11. Errores comunes y cómo resolverlos

1. Mapa de la fase

El objetivo es responder: ¿cómo voy a almacenar, consultar y presentar la información? El recorrido completo es:

Etapas	Qué ocurre	Herramienta	Archivo del paquete
1. Dataset	Descargar los 9 CSV de Kaggle	Navegador	data/raw/
2. Limpieza	Corregir tipos, nulos, duplicados, categorías	Python (pandas)	python/etlolist.py
3. PostgreSQL	Levantar la base de datos aislada	Docker	docker-compose.yml
4. Staging	Cargar los datos limpios tal cual	Python + SQL	sql/01_staging.sql
5. Modelo	Crear tablas de hechos, dimensiones, PK y FK	SQL	sql/02_modelo_dimensional.sql
6. Carga	Llenar dimensiones y hechos	SQL	sql/03_carga_dimensiones_hechos.sql
7. Validación	Comprobar conteos, integridad, nulos y totales	SQL	sql/04_validaciones.sql
8. Vistas/funciones	Objetos que alimentarán el dashboard	SQL	sql/05_vistas_funciones.sql
9. Mockup	Diseño del dashboard + trazabilidad	Imágenes	mockup/ y este documento

Regla de oro: si algo falla, no avances. Cada etapa depende de la anterior (no se pueden crear claves foráneas sin tener antes las dimensiones, ni cargar hechos sin dimensiones).

2. Teoría esencial

2.1 PostgreSQL, SQL y esquemas

PostgreSQL es un motor de base de datos relacional: guarda la información en tablas (filas y columnas) y se consulta con SQL. Dentro de una base de datos se pueden crear esquemas, que son carpetas lógicas para ordenar las tablas. En este proyecto usamos dos: stg (staging, datos limpios recién llegados) y dw (data warehouse, el modelo dimensional final).

Término	Significado	Ejemplo en el proyecto
DDL	Instrucciones que definen estructura	CREATE TABLE dw.dim_cliente (...)
DML	Instrucciones que manipulan datos	INSERT INTO ... SELECT ..., TRUNCATE
PK (clave primaria)	Columna(s) que identifican de forma única cada fila	cliente_key; (order_id, order_item_id)
FK (clave foránea)	Columna que apunta a la PK de otra tabla y obliga a que exista	fact.cliente_key → dim_cliente.cliente_key
Índice	Estructura que acelera búsquedas	ix_fact_fecha_compra
Vista	Consulta guardada con nombre; se usa como una tabla	dw.v_ventas_mensual
Función	Consulta guardada que recibe parámetros	dw.fn_total_ventas(desde, hasta, ...)

2.2 Docker y docker compose

Docker permite ejecutar programas dentro de contenedores: entornos aislados y reproducibles. En lugar de instalar PostgreSQL en tu sistema, descargas una imagen (postgres:17) y Docker crea un contenedor (bi-postgres) a partir de ella. El archivo docker-compose.yml describe cómo debe ser ese contenedor, y con un solo comando lo enciendes o lo apagas.

Línea del docker-compose.yml	Qué hace
image: postgres:17	Usa la imagen oficial de PostgreSQL versión 17
container_name: bi-postgres	Nombre del contenedor de la base (lo usarás en los comandos docker exec)
POSTGRES_USER / PASSWORD / DB	Crea el usuario bi_user, su contraseña y la base bi_database la primera vez
ports: \${DB_PORT:-5432}:5432	Conecta un puerto de tu PC (5432 por defecto; configurable en .env) con el 5432 del contenedor (host: localhost)
volumes: postgres_data	Disco persistente: tus datos NO se pierden al apagar con docker compose down
./sql:/scripts:ro	Comparte la carpeta sql con el contenedor para poder ejecutar scripts a mano con docker exec

docker compose down **conserva** los datos; docker compose down -v **borra** también el volumen (empiezas de cero).

2.3 Sistema operacional (OLTP) vs analítico (OLAP)

Los CSV de Olist vienen normalizados en 9 tablas, como en un sistema que registra ventas (OLTP): eso es eficiente para guardar pedidos, pero incómodo para analizar, porque cada

pregunta exige muchos JOIN. Un modelo dimensional reorganiza esos datos para responder preguntas de análisis (OLAP) de forma simple y rápida: sumar, promediar y contar por mes, categoría, estado o vendedor.

2.4 Modelo dimensional

Concepto	Definición	En tu modelo
Tabla de hechos	Registra eventos de negocio medibles. Es la tabla grande, con medidas y claves foráneas	fact_ventas_item
Dimensión	Describe el contexto de los hechos (quién, qué, cuándo, dónde). Se usa para filtrar y agrupar	dim_fecha, dim_cliente, dim_producto, dim_vendedor, dim_estado_pedido, dim_pago
Medida	Valor numérico que se analiza	price, freight_value, dias_entrega, review_score
Granularidad	Qué representa UNA fila de la tabla de hechos. Se define primero; todo depende de ella	Un ítem vendido dentro de un pedido
Esquema estrella	Hechos al centro y dimensiones alrededor, unidas solo por claves	Ver diagrama (sección 3)
Clave sustituta (surrogate)	Número artificial creado por nosotros como PK de la dimensión	cliente_key, producto_key...
Clave de negocio	Identificador original del dato	customer_unique_id, product_id, seller_id
Dimensión degenerada	Identificador que vive en la tabla de hechos porque no tiene atributos propios	order_id
Dimensión de rol	Una misma dimensión usada con distintos significados	dim_fecha como fecha de compra y de entrega
Dimensión basura (junk)	Agrupar atributos pequeños y sueltos en una sola dimensión	dim_pago (tipo de pago + grupo de cuotas)

Tipos de medidas

- **Aditivas:** se pueden sumar en cualquier dimensión (price, freight_value, cantidad).
- **No aditivas / derivadas:** se promedian o se recalculan, nunca se suman (dias_entrega, dias_retraso, entrega_a_tiempo, review_score).

Trampa típica de este dataset: la reseña y los días de entrega son datos del PEDIDO, pero la tabla de hechos tiene una fila por ÍTEM. Un pedido con 3 ítems repetirá su review_score 3 veces. Si lo promedias directamente, los pedidos grandes pesan más de lo debido. Solución aplicada: las funciones SQL agrupan primero por order_id (una fila por pedido) y recién después calculan promedios o porcentajes. Lo mismo con el ticket promedio: ingresos ÷ pedidos únicos.

2.5 ETL y staging

ETL = Extract (leer los CSV), Transform (limpiar y transformar), Load (cargar). Se usa una zona intermedia llamada staging: primero se cargan datos limpios en tablas simples (esquema stg) y luego, con SQL, se construyen dimensiones y hechos. Así, si algo sale mal en el modelo, no hay que volver a limpiar los CSV.

2.6 Vistas vs funciones

	Vista	Función
Parámetros	No	Sí (fechas, categoría, estado...)
Uso típico	Gráficos y tablas fijas	KPI y gráficos que dependen de los filtros del dashboard
Llamada	SELECT * FROM dw.v_ventas_mensual;	SELECT dw.fn_total_ventas('2017-01-01','2017-12-31');

En las funciones, un parámetro en NULL significa “sin filtro” (todos). Así un mismo objeto SQL sirve tanto para el total general como para el filtrado.

2.7 Trazabilidad

La matriz de trazabilidad demuestra que cada elemento visual del dashboard (tarjeta, gráfico, tabla) se alimenta de un objeto SQL concreto, que a su vez sale de las tablas del modelo y responde a una pregunta de negocio de tu Entregable 1. Es la cadena: Pregunta/KPI → Componente → Filtros → Origen de datos → Objeto SQL.

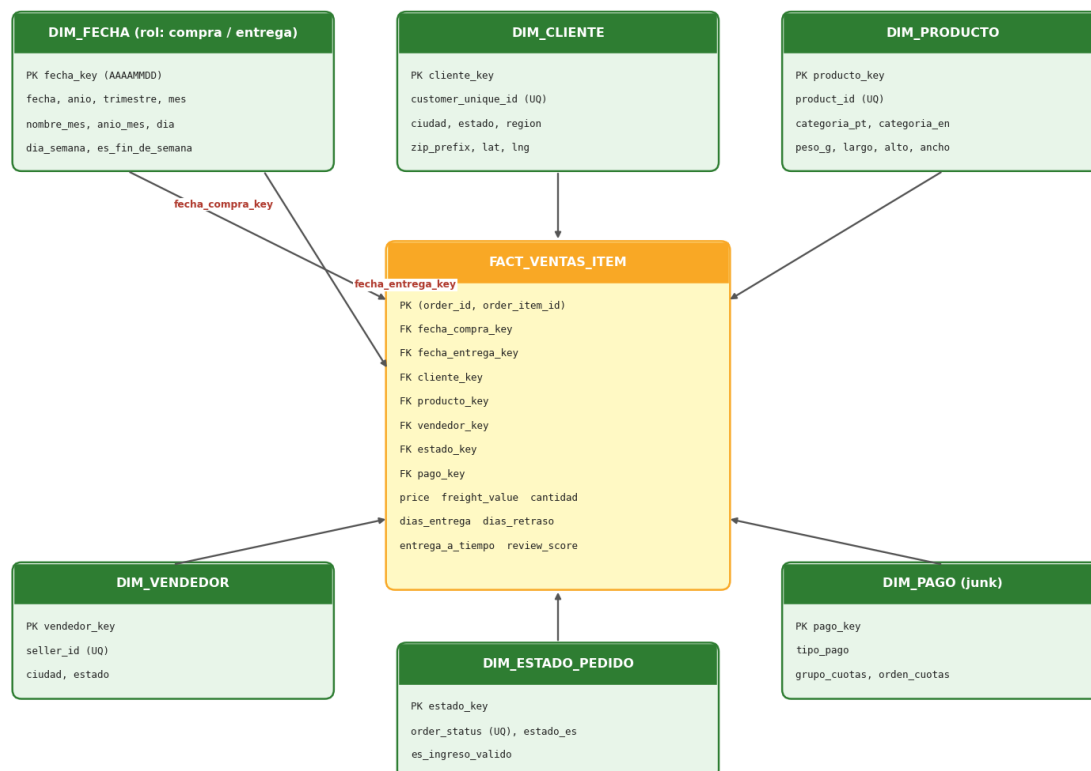
3. Modelo dimensional definitivo

3.1 Proceso de negocio y granularidad

Proceso de negocio: venta de productos en el marketplace Olist (desde la compra hasta la entrega y la reseña).

Granularidad (frase obligatoria): cada fila de fact_ventas_item representa un ítem (producto) vendido dentro de un pedido, identificado por la combinación order_id + order_item_id.

3.2 Diagrama estrella



Esquema estrella - Granularidad: una fila = un ítem (producto) vendido dentro de un pedido

3.3 Tabla de hechos: dw.fact_ventas_item

Columna	Tipo	Rol	Descripción
order_id	VARCHAR(32)	PK / dim. degenerada	Identificador del pedido
order_item_id	SMALLINT	PK	Posición del ítem dentro del pedido
fecha_compra_key	INTEGER	FK → dim_fecha	Fecha de compra (AAAAMMDD)
fecha_entrega_key	INTEGER	FK → dim_fecha	Fecha de entrega; NULL si no fue entregado
cliente_key / producto_key / vendedor_key	INTEGER	FK	Claves hacia las dimensiones
estado_key / pago_key	SMALLINT	FK	Estado del pedido y modalidad de pago
price	NUMERIC(10,2)	Medida aditiva	Ingreso del ítem (KPI ingresos)
freight_value	NUMERIC(10,2)	Medida aditiva	Flete del ítem
cantidad	SMALLINT	Medida aditiva	Siempre 1 (conteo de ítems)
dias_entrega	NUMERIC(8,2)	Derivada	Días entre compra y entrega
dias_retraso	SMALLINT	Derivada	Días tarde respecto a la fecha estimada (0 = a tiempo)
entrega_a_tiempo	SMALLINT	Derivada	1 si entregó en la fecha estimada o antes, 0 si no
review_score	SMALLINT	Derivada (nivel pedido)	Calificación 1–5; promediar por pedido

3.4 Dimensiones

Dimensión	Clave (PK)	Clave de negocio	Atributos principales	Jerarquía
dim_fecha	fecha_key (AAAAMMDD)	fecha	año, trimestre, mes, nombre_mes, año_mes, día, nombre_día, es_fin_de_semana	Día → Mes → Trimestre → Año
dim_cliente	cliente_key	customer_unique_id	ciudad, estado, region, zip_prefix, lat, lng	Ciudad → Estado → Región
dim_vendedor	vendedor_key	seller_id	ciudad, estado	Ciudad → Estado
dim_producto	producto_key	product_id	categoría_pt, categoría_en, peso_g, largo, alto, ancho	Producto → Categoría
dim_estado_pedido	estado_key	order_status	estado_es, es_ingreso_valido	—
dim_pago	pago_key	(tipo_pago, grupo_cuotas)	tipo_pago, grupo_cuotas	—

3.5 Decisiones de diseño

Tema	Decisión	Justificación
Cliente	La dimensión tiene una fila por customer_unique_id (persona real), no por customer_id	customer_id cambia en cada pedido; si se usara, la tasa de recompra siempre sería 0. Si la persona cambió de ciudad se conserva la ubicación de su compra más reciente.
Pagos	Se resumen a UNA fila por pedido (tipo de mayor valor + máximo de cuotas) y se modelan como dim_pago	Un pedido puede tener varios pagos; unirlos directo al ítem multiplicaría los montos. Los montos de ingreso salen de price, no de payment_value.
Reseñas	Una reseña por pedido (la más reciente); se guarda como review_score en los hechos	review_id aparece repetido en algunos casos; así no se duplican filas. El texto de los comentarios se excluye (muy vacío).
Fechas	dim_fecha con dos roles; fecha_entrega_key admite NULL	Un pedido no entregado no tiene fecha de entrega; es un nulo real y se valida, no se inventa una fecha.
Estado	Se conservan todos los estados y se marca es_ingreso_valido	Permite que el KPI excluya cancelados/no disponibles sin borrar datos.
Período	Solo compras entre 2017-01-01 y 2018-08-31	2016 y sep-oct 2018 casi no tienen datos y distorsionan tendencias (Entregable 1).
Segunda tabla de hechos	No se crea fact_pedido; se resuelve con agrupación por order_id en las funciones	Cumple el mínimo (1 hecho + 3 dimensiones) con menos complejidad. Es una decisión que puedes justificar y ampliar después.

4. Transformaciones realizadas sobre el dataset

Cada transformación la ejecuta python/etl_olist.py y queda registrada con sus conteos de filas antes y después en docs/log_transformaciones.csv (esa es tu evidencia de que se documentó).

#	Tabla	Transformación	Motivo (problema del Entregable 1)
1	orders	Fechas de texto → datetime	Tipos incorrectos
2	orders	Eliminar order_id duplicados	Duplicados
3	orders	Filtrar compras 2017-01-01 a 2018-08-31	Cobertura temporal
4	orders	Descartar entregas anteriores a la compra	Fechas incoherentes
5	orders	Estado normalizado; nulos de fechas se mantienen	Nulos en pedidos no completados
6	items	Quedarse con ítems de pedidos del período; fecha a datetime	Integridad entre tablas
7	items	Detectar atípicos de price con IQR; se conservan	Valores atípicos legítimos
8	customers	Ciudad en minúsculas sin acentos; zip como texto; solo clientes con pedidos	Categorías inconsistentes / ceros iniciales
9	sellers	Ciudad normalizada; zip como texto	Categorías inconsistentes
10	products	Unir con traducción; faltantes → sin_categoria; 2 traducciones manuales; medidas 0 → NULL; columnas renombradas	Nulos, categorías sin traducción, nombres con errores ortográficos
11	payments	Resumir a una fila por pedido; tipos inválidos → not_defined	Pagos múltiples / not_defined
12	reviews	Conservar la reseña más reciente por pedido; excluir comentarios	Duplicados / columnas poco útiles

#	Tabla	Transformación	Motivo (problema del Entregable 1)
13	geolocation	Una fila por código postal con promedio de lat/lng; coordenadas fuera de Brasil descartadas	Cientos de miles de duplicados

Transformaciones posteriores dentro de PostgreSQL (script 03): generación de dim_fecha, asignación de región según el estado, agrupación de cuotas (1 pago, 2-3, 4-6, 7-12, más de 12) y cálculo de dias_entrega, dias_retraso y entrega_a_tiempo.

5. Consultas de validación

Qué valida	Consulta	Resultado esperado
Registros cargados	COUNT(*) por tabla (UNION ALL)	Dimensiones y hechos con filas; dim_fecha = 1461; dim_estado_pedido = 8; dim_pago = 25
Conciliación staging ↔ hechos	Ítems en staging vs filas de fact	Diferencia = 0
Integridad referencial	LEFT JOIN de hechos contra cada dimensión	Todos los contadores sin_* = 0
Claves duplicadas	GROUP BY ... HAVING COUNT(*) > 1	0 filas
Restricciones declaradas	Consulta a pg_constraint	Lista de PK y FK del modelo
Nulos relevantes	COUNT FILTER (WHERE col IS NULL)	price y freight_value sin nulos; nulos de entrega = pedidos no entregados; pocos sin reseña
Coherencia estado vs entrega	Agrupar por order_status	Los 'delivered' no tienen fecha de entrega nula
Valores imposibles	Negativos, entrega antes de compra, score fuera de 1-5	Todo en 0
Totales principales	Pedidos, ítems, ingresos, flete, ticket, fechas mín/máx	Fechas entre 2017-01-01 y 2018-08-31
Conciliación de montos	SUM(price) staging vs hechos	Diferencia = 0.00
Ingresos por mes	Agrupar por anio_mes	20 meses (2017-01 a 2018-08)

6. Vistas y funciones SQL

Definidas en sql/05_vistas_funciones.sql. Regla de ingresos: SUM (price) de pedidos cuyo estado no sea cancelado ni no disponible.

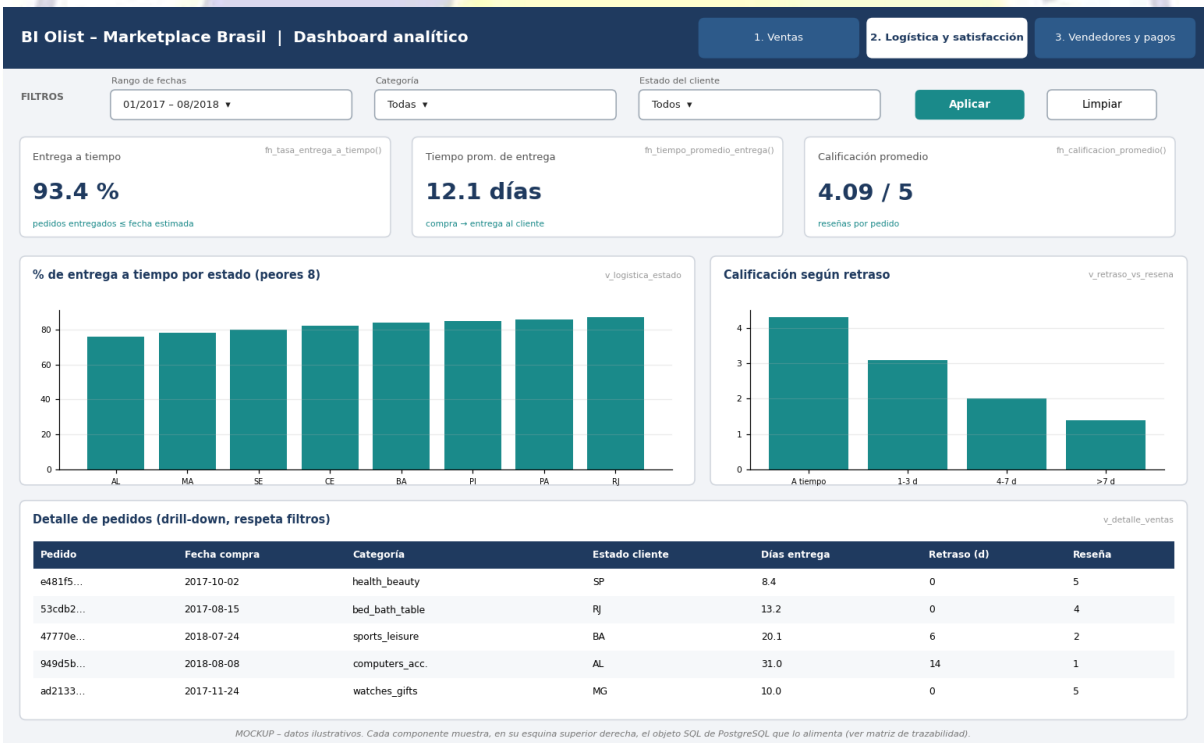
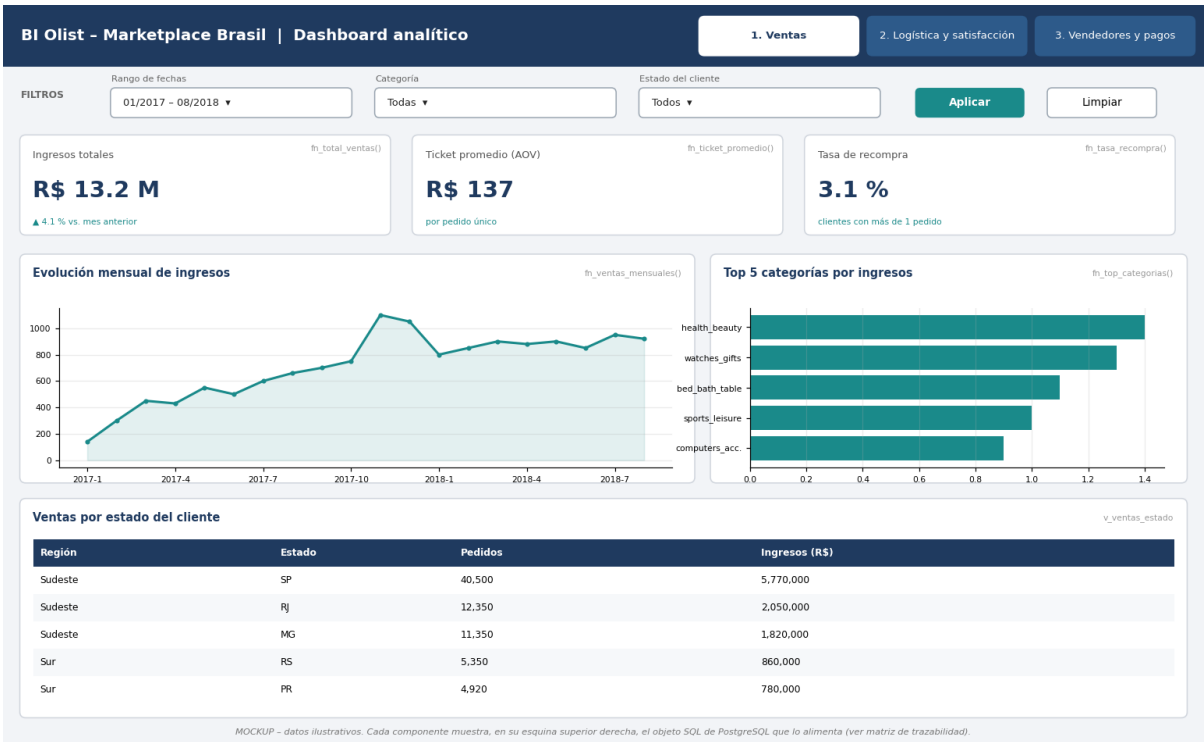
Objeto	Tipo	Devuelve	Parámetros / filtros
fn_base_pedidos	Función de tabla	Una fila por pedido con ingresos, flete, entrega y reseña	desde, hasta, categoría, estado cliente
fn_total_ventas	Función escalar	Ingresos totales	desde, hasta, categoría, estado cliente

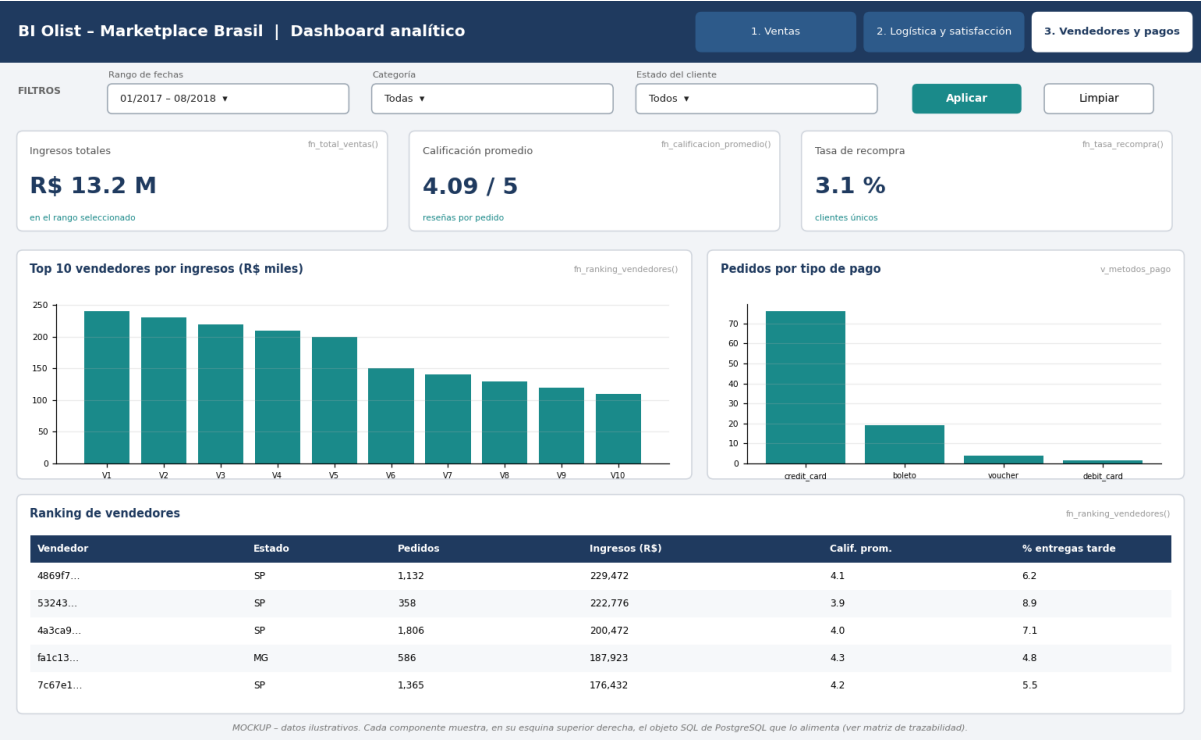
Objeto	Tipo	Devuelve	Parámetros / filtros
fn_ticket_promedio	Función escalar	Ingresos ÷ pedidos únicos	ídem
fn_tasa_entrega_a_tiempo	Función escalar	% de pedidos entregados a tiempo	ídem
fn_tiempo_promedio_entrega	Función escalar	Días promedio de entrega	ídem
fn_calificacion_promedio	Función escalar	Promedio de review_score por pedido	ídem
fn_tasa_recompra	Función escalar	% de clientes con más de un pedido	ídem
fn_ventas_mensuales	Función de tabla	Pedidos, ingresos y ticket por mes	ídem
fn_top_categorias	Función de tabla	Top N categorías por ingresos	desde, hasta, estado cliente, N
fn_ranking_vendedores	Función de tabla	Ingresos, calificación y % de entregas tarde por vendedor	desde, hasta, estado vendedor, N
v_ventas_mensual	Vista	Ventas por mes (todo el período)	—
v_ventas_categoria	Vista	Ingresos por categoría	—
v_ventas_estado	Vista	Ingresos por región y estado del cliente	—
v_logistica_estado	Vista	Entrega a tiempo y días por estado	—
v_retrazo_vs_resena	Vista	Calificación según rango de retraso (P3)	—
v_metodos_pago	Vista	Pedidos e ingresos por tipo de pago y cuotas (P5)	—
v_detalle_ventas	Vista	Detalle plano por ítem (tabla drill-down)	Se filtra por columnas (fecha, categoría, estado)

7. Mockup del dashboard

El dashboard se organiza en tres vistas (pestañas). Todas comparten una barra de filtros (rango de fechas, categoría y estado del cliente) y cumplen el mínimo pedido: 3 KPI, filtros, 2 gráficos y una tabla de detalle por vista.

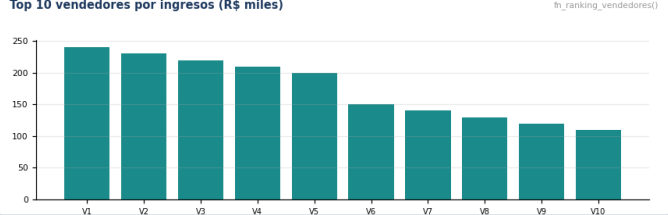
Vista	Pregunta que responde	KPI	Gráficos	Detalle
1. Ventas	P1, P5	Ingresos, ticket promedio, recompra	Línea mensual; Top categorías	Ventas por estado/región
2. Logística y satisfacción	P2, P3	% a tiempo, días de entrega, calificación	% a tiempo por estado; calificación vs retraso	Detalle de pedidos
3. Vendedores y pagos	P4, P5	Ingresos, calificación, recompra	Top vendedores; tipos de pago	Ranking de vendedores





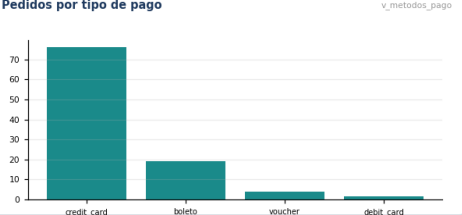
Top 10 vendedores por ingresos (R\$ miles)

fn_ranking_vendedores()



Pedidos por tipo de pago

v_metodos_pago



Ranking de vendedores

fn_ranking_vendedores()

Vendedor	Estado	Pedidos	Ingresos (R\$)	Calif. prom.	% entregas tarde
4869f7...	SP	1,132	229,472	4.1	6.2
53243...	SP	358	222,776	3.9	8.9
4a3ca9...	SP	1,806	200,472	4.0	7.1
fa1c13...	MG	586	187,923	4.3	4.8
7c67e1...	SP	1,365	176,432	4.2	5.5

MOCKUP - datos ilustrativos. Cada componente muestra, en su esquina superior derecha, el objeto SQL de PostgreSQL que lo alimenta (ver matriz de trazabilidad).

8. Matriz de trazabilidad

Mockup → Pregunta/KPI → Filtros → Origen de datos → Objeto SQL. (Filtros: F = fecha, C = categoría, E = estado del cliente.)

Componente	Pregunta / KPI	Filtros	Origen de datos	Objeto SQL
Tarjeta K1: Ingresos totales	P1 · Ingresos por ventas	F, C, E	Fact + dim_fecha, dim_producto, dim_cliente, dim_estado_pedido	dw.fn_total_ventas()
Tarjeta K2: Ticket promedio	P1, P5 · Ticket promedio (AOV)	F, C, E	Fact + dimensiones (pedidos únicos)	dw.fn_ticket_promedio()
Tarjeta K3: Tasa de recompra	P5 · Tasa de recompra	F, C, E	Fact + dim_cliente (customer_unique_id)	dw.fn_tasa_recompra()
Gráfico G1: Línea de ingresos mensuales	P1 · Evolución de ventas por mes	F, C, E	Fact + dim_fecha	dw.fn_ventas_mensuales()
Gráfico G2: Barras Top categorías	P1 · Categorías con más ingresos	F, E	Fact + dim_producto	dw.fn_top_categorias()
Tabla T1: Ventas por estado/región	P1 · Estados que generan más ingresos	—	Fact + dim_cliente	dw.v_ventas_estado

Componente	Pregunta / KPI	Filtros	Origen de datos	Objeto SQL
Tarjeta K4: Entrega a tiempo	P2, P3 · Tasa de entrega a tiempo	F, C, E	Fact (entrega_a_tiempo) + dimensiones	dw.fn_tasa_entrega_a_tiempo()
Tarjeta K5: Tiempo prom. de entrega	P2 · Tiempo promedio de entrega	F, C, E	Fact (dias_entrega) + dimensiones	dw.fn_tiempo_promedio_entrega()
Tarjeta K6: Calificación promedio	P3, P4 · Calificación promedio	F, C, E	Fact (review_score) + dimensiones	dw.fn_calificacion_promedio()
Gráfico G3: % a tiempo por estado	P2 · Estados con más retrasos	—	Fact + dim_cliente	dw.v_logistica_estado
Gráfico G4: Calificación vs retraso	P3 · Efecto del retraso en la satisfacción	—	Fact (dias_retraso, review_score)	dw.v_retraso_vs_resena
Tabla T2: Detalle de pedidos	P2, P3 · Drill-down de pedidos	F, C, E	Fact + todas las dimensiones	dw.v_detalle_ventas
Gráfico G5: Pedidos por tipo de pago	P5 · Medios de pago preferidos	—	Fact + dim_pago	dw.v_metodos_pago
Gráfico G6: Top vendedores	P4 · Vendedores que concentran ingresos	F	Fact + dim_vendedor	dw.fn_ranking_vendedores()
Tabla T3: Ranking de vendedores	P4 · Ingresos y reputación por vendedor	F, estado vendedor	Fact + dim_vendedor	dw.fn_ranking_vendedores()
Filtros globales (fecha, categoría, estado)	Todos	—	dim_fecha, dim_producto, dim_cliente	Parámetros de las funciones fn_*

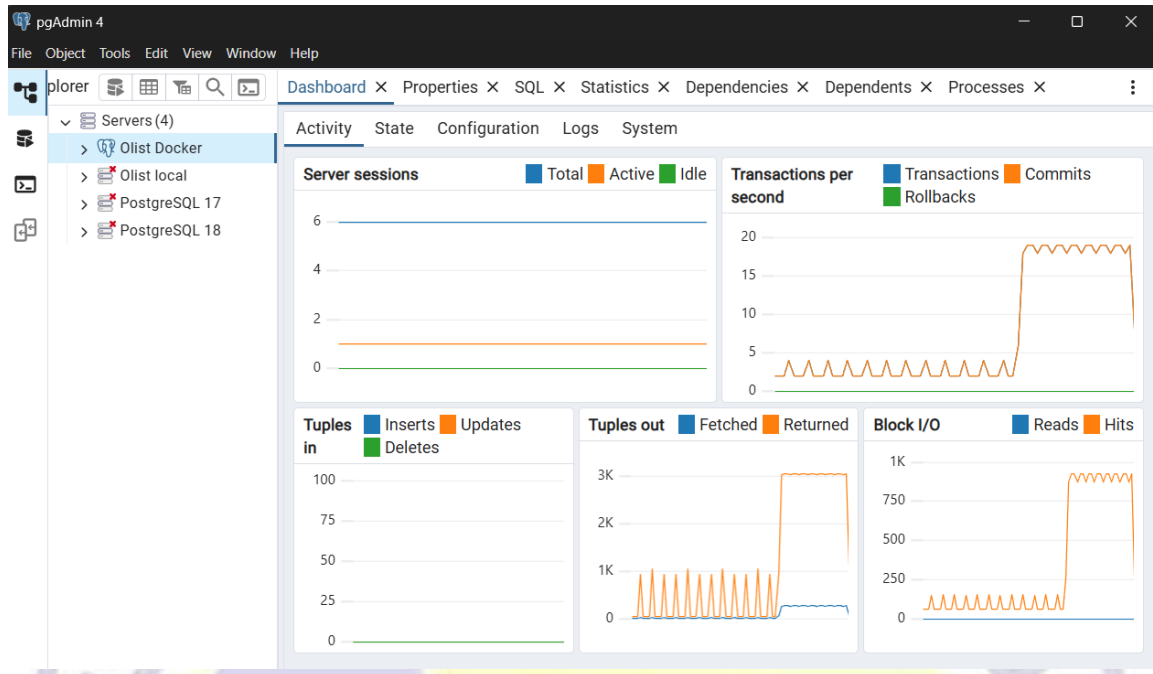
Ejemplo de uso en SQL (lo que ejecutaría la aplicación al aplicar los filtros 2017, categoría health_beauty, estado SP):

```
SELECT dw.fn_total_ventas('2017-01-01', '2017-12-31', 'health_beauty', 'SP');
SELECT * FROM dw.fn_ventas_mensuales('2017-01-01', '2017-12-31', 'health_beauty', 'SP');
```

9. Evidencias

```
PS C:\Users\Usuario\Downloads\Fase2_BI_Olist_Docker_GitHub\Fase2_BI_Oli
st> docker compose run --rm etl
[+] run 3/3
  ✓ Network bi-olist_default      Created      0.1s
  ✓ Volume bi-olist_postgres_data Created      0.0s
  ✓ Container bi-postgres         Created      0.2s
Image bi-olist-etl Building
[+] Building 69.9s (10/10) FINISHED
=> [internal] load local bake definitions      0.2s
=> => reading from stdin 619B                  0.2s
=> [internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 329B             0.0s
=> [internal] load metadata for docker.io/library/python:3.12-s 2.1s
=> [internal] load .dockerignore               0.0s
=> => transferring context: 2B                  0.0s
=> [1/4] FROM docker.io/library/python:3.12-slim@sha256:02108f5 5.5s
=> => resolve docker.io/library/python:3.12-slim@sha256:02108f5 0.1s
=> => sha256:5e62c0470a0973ad9561440c8a37984c3252ed 249B / 249B 0.2s
=> => sha256:e360d85fdc0f3910a4fe48ba2bb0865d 12.12MB / 12.12MB 2.3s
=> => sha256:a510a9ea6d60786b7cefdbf9577945344c 4.27MB / 4.27MB 3.7s
=> => extracting sha256:a510a9ea6d60786b7cefdbf9577945344c72879 0.3s
=> => extracting sha256:e360d85fdc0f3910a4fe48ba2bb0865d5e61839 1.1s
=> => extracting sha256:5e62c0470a0973ad9561440c8a37984c3252eda 0.0s
=> [2/4] RUN apt-get update && apt-get install -y --no-instal 13.9s
=> [3/4] RUN pip install --no-cache-dir pandas psycpg2-binary 29.7s
=> [4/4] WORKDIR /app                                           0.2s
=> exporting to image                                           17.6s
=> => exporting layers                                          11.5s
=> => exporting manifest sha256:38f5e489e2408b4a201c12a2d838076 0.0s
=> => exporting config sha256:025da9e52eb1ab6c03a3af0eab8641a55 0.0s
```

```
Conectando a PostgreSQL ...
stg.pedidos          99,092 filas cargadas
stg.items            112,279 filas cargadas
stg.clientes         99,092 filas cargadas
stg.vendedores       3,095 filas cargadas
stg.productos        32,951 filas cargadas
stg.pagos_pedido     99,092 filas cargadas
stg.resenas          98,330 filas cargadas
stg.geolocalizacion  19,010 filas cargadas
Staging cargado correctamente.
=== 3/6 Modelo dimensional (hechos, dimensiones, PK y FK) ===
psql:sql/02_modelo_dimensional.sql:13: NOTICE: table "fact_ventas_item
" does not exist, skipping
psql:sql/02_modelo_dimensional.sql:13: NOTICE: table "dim_fecha" does
not exist, skipping
psql:sql/02_modelo_dimensional.sql:13: NOTICE: table "dim_cliente" doe
s not exist, skipping
psql:sql/02_modelo_dimensional.sql:13: NOTICE: table "dim_producto" do
es not exist, skipping
psql:sql/02_modelo_dimensional.sql:13: NOTICE: table "dim_vendedor" do
es not exist, skipping
psql:sql/02_modelo_dimensional.sql:13: NOTICE: table "dim_estado_pedid
o" does not exist, skipping
psql:sql/02_modelo_dimensional.sql:13: NOTICE: table "dim_pago" does n
ot exist, skipping
=== 4/6 Carga de dimensiones y tabla de hechos ===
=== 5/6 Validaciones ===
-> evidencias/validaciones.txt
=== 6/6 Vistas, funciones y pruebas ===
```



Query Query History

1 SELECT * FROM dw.dim_cliente
2 ORDER BY cliente_key ASC

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 96

	cliente_key [PK] integer	customer_unique_id character varying (32)	ciudad character varying (100)	estado character (2)	region character varying (20)	zip_prefix character varying (5)	lat numeric (10,6)	lng numeric (10,6)
1	1	0000366f3b9a7992bf8e76cfd3221e2	cajamar	SP	Sudeste	07787	-23.340235	-46.830140
2	2	0000b849f77a9e4a4ce2b2a4ca5be...	osasco	SP	Sudeste	06053	-23.559115	-46.787626
3	3	0000f46a3911fa3c08054444833370...	sao jose	SC	Sur	88115	-27.542880	-48.633426
4	4	0000f6cc0745a6a4b88665a16c9f0...	belem	PA	Norte	66812	-1.312214	-48.483159
5	5	0004aac84e0df4da2b147fca70cf8255	sorocaba	SP	Sudeste	18040	-23.505548	-47.469705
6	6	0004bd2a26a76fe21f786e4bd80607f	sao paulo	SP	Sudeste	05036	-23.515676	-46.688233
7	7	00050ab1314c0e55a6ca13cf7181fecf	campinas	SP	Sudeste	13084	-22.823421	-47.087193
8	8	00053a61a98854899e70ed204dd4b...	curitiba	PR	Sur	80410	-25.431323	-49.279219
9	9	0005e1862207bf6ccc02e4228effd9a0	teresopolis	RJ	Sudeste	25966	-22.401371	-42.991002
10	10	0005ef4cd20d2893f0d9fb94d3cd997	sao luis	MA	Nordeste	65060	-2.545261	-44.234993
11	11	0006fde98a402fceb4eb0e528f6a8d4	mimoso do sul	ES	Sudeste	29400	-21.061571	-41.362964
12	12	00082cbe03e478190aadbea78542e...	itapeva	SP	Sudeste	18400	-23.983249	-48.876525
13	13	00090324bbad0e9342388303bb71b...	campinas	SP	Sudeste	13054	-22.974131	-47.126450
14	14	000949456b182f53c18b68d6babc79...	sao bernardo do campo	SP	Sudeste	09613	-23.670915	-46.565135
15	15	000a5ad9c4601d2bdd9ed765d521...	porto alegre	RS	Sur	90560	-30.020689	-51.200706
16	16	000bfa1d2f1a41876493be685390d6...	santos	SP	Sudeste	11095	-23.927266	-46.378475
17	17	000c8bdb58a29e7115cfc257230fb2...	belo horizonte	MG	Sudeste	31555	-19.841601	-43.980222
18	18	000d460961d6dbfa3ec5c9f5805769...	sao paulo	SP	Sudeste	01206	-23.535864	-46.642819

Query Query History

1 SELECT * FROM dw.dim_estado_pedido
2 ORDER BY estado_key ASC

Data Output Messages Notifications

	estado_key [PK] smallint	order_status character varying (20)	estado_es character varying (30)	es_ingreso_valido boolean
1	1	delivered	Entregado	true
2	2	shipped	Enviado	true
3	3	invoiced	Facturado	true
4	4	processing	En proceso	true
5	5	approved	Aprobado	true
6	6	created	Creado	true
7	7	canceled	Cancelado	false
8	8	unavailable	No disponible	false

- FTS Templates
- Foreign Tables
- Functions
- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (7)
 - dim_cliente
 - dim_estado_pedido
 - dim_fecha
 - dim_pago
 - dim_producto
 - dim_vendedor
 - fact_ventas_item
- Trigger Functions
- Types
- Views
- public
- stg
- Subscriptions
- postgres
- login/Group Roles
- tablespaces
- local

1 **SELECT** * **FROM** dw.dim_fecha
2 **ORDER BY** fecha_key **ASC**

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 2

	fecha_key [PK] integer	fecha date	anio smallint	trimestre smallint	mes smallint	nombre_mes character varying (12)	anio_mes character (7)	dia smallint	dia_semana_num smallint	nombre_dia character varying (12)	es_fin_de_semana boolean
1	20160101	2016-01-01	2016	1	1	Enero	2016-01	1	5	Viernes	false
2	20160102	2016-01-02	2016	1	1	Enero	2016-01	2	6	Sábado	true
3	20160103	2016-01-03	2016	1	1	Enero	2016-01	3	7	Domingo	true
4	20160104	2016-01-04	2016	1	1	Enero	2016-01	4	1	Lunes	false
5	20160105	2016-01-05	2016	1	1	Enero	2016-01	5	2	Martes	false
6	20160106	2016-01-06	2016	1	1	Enero	2016-01	6	3	Miércoles	false
7	20160107	2016-01-07	2016	1	1	Enero	2016-01	7	4	Jueves	false
8	20160108	2016-01-08	2016	1	1	Enero	2016-01	8	5	Viernes	false
9	20160109	2016-01-09	2016	1	1	Enero	2016-01	9	6	Sábado	true
10	20160110	2016-01-10	2016	1	1	Enero	2016-01	10	7	Domingo	true
11	20160111	2016-01-11	2016	1	1	Enero	2016-01	11	1	Lunes	false
12	20160112	2016-01-12	2016	1	1	Enero	2016-01	12	2	Martes	false
13	20160113	2016-01-13	2016	1	1	Enero	2016-01	13	3	Miércoles	false
14	20160114	2016-01-14	2016	1	1	Enero	2016-01	14	4	Jueves	false
15	20160115	2016-01-15	2016	1	1	Enero	2016-01	15	5	Viernes	false
16	20160116	2016-01-16	2016	1	1	Enero	2016-01	16	6	Sábado	true
17	20160117	2016-01-17	2016	1	1	Enero	2016-01	17	7	Domingo	true

- FTS Templates
- Foreign Tables
- Functions
- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (7)
 - dim_cliente
 - dim_estado_pedido
 - dim_fecha
 - dim_pago
 - dim_producto
 - dim_vendedor
 - fact_ventas_item
- Trigger Functions
- Types
- Views
- public
- stg
- Subscriptions
- postgres
- login/Group Roles
- tablespaces
- local

Query Query History

1 **SELECT** * **FROM** dw.dim_pago
2 **ORDER BY** pago_key **ASC**

Data Output Messages Notifications

	pago_key [PK] smallint	tipo_pago character varying (20)	grupo_cuotas character varying (20)	orden_cuotas smallint
1	1	credit_card	1 pago	1
2	2	credit_card	2-3 cuotas	2
3	3	credit_card	4-6 cuotas	3
4	4	credit_card	7-12 cuotas	4
5	5	credit_card	Más de 12 cuotas	5
6	6	boleto	1 pago	1
7	7	boleto	2-3 cuotas	2
8	8	boleto	4-6 cuotas	3
9	9	boleto	7-12 cuotas	4
10	10	boleto	Más de 12 cuotas	5
11	11	voucher	1 pago	1
12	12	voucher	2-3 cuotas	2
13	13	voucher	4-6 cuotas	3
14	14	voucher	7-12 cuotas	4
15	15	voucher	Más de 12 cuotas	5
16	16	debit_card	1 pago	1
17	17	debit_card	2-3 cuotas	2
18	18	debit_card	4-6 cuotas	3

- Procedures
- Sequences
- Tables (7)
 - dim_cliente
 - dim_estado_pedido
 - dim_fecha
 - dim_pago
 - dim_producto
 - dim_vendedor
 - fact_ventas_item
- Trigger Functions
- Types
- Views
- public
- stg
- Subscriptions
- postgres
- login/Group Roles
- tablespaces
- local

	producto_key [PK] integer	product_id character varying (32)	categoria_pt character varying (100)	categoria_en character varying (100)	peso_g numeric (10,2)	largo_cm numeric (8,2)	alto_cm numeric
1	1	1e9e8ef04dbcf4541ed26657ea517e5	perfumaria	perfumery	225.00	16.00	
2	2	3aa071139cb16b67ca9e5dea641aaa2f	artef	art	1000.00	30.00	
3	3	96bd76ec8810374ed1b65e29197571...	esporte_lazer	sports_leisure	154.00	18.00	
4	4	cef67bce19066a932b7673e239eb23d	bebes	baby	371.00	26.00	
5	5	9dc1a7de27444849c219cf195d0b71	utilidades_domesticas	housewares	625.00	20.00	
6	6	41d3672d4792049fa1779bb35283ed...	instrumentos_musicales	musical_instruments	200.00	38.00	
7	7	732bd381ad09e530fe0a5f457d81becb	cool_stuff	cool_stuff	18350.00	70.00	
8	8	2548af3fe77a690cf3eb6368e9ab51e	moveis_decoracao	furniture_decor	900.00	40.00	
9	9	37cc742be07708b53a98702e77a21a...	eletrodomesticos	home_appliances	400.00	27.00	
10	10	8c92109888e8cdf9d66dc7e463025574	brinquedos	toys	600.00	17.00	
11	11	14aa47b7fe5c25522b47b4b29c98dcb9	cama_mesa_banho	bed_bath_table	1100.00	16.00	
12	12	03b63c5fc16691530586ae020c345514	bebes	baby	7150.00	50.00	
13	13	cf55509ea8edaac1d28fdb16e48fc22	instrumentos_musicales	musical_instruments	250.00	17.00	
14	14	7bb6f29c2be5771619496496660c7c2	moveis_decoracao	furniture_decor	600.00	68.00	
15	15	eb31436580a610f202c859463d8c74...	construcao_ferramentas_seguranca	construction_tools_safety	200.00	17.00	
16	16	3bb71144022e6732727d8d838a7b13...	esporte_lazer	sports_leisure	800.00	16.00	
17	17	6a2fb4dd53d2c8b88e0432f1284a004c	perfumaria	perfumery	400.00	27.00	

- FTS Templates
- Foreign Tables
- Functions
- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (7)
 - dim_cliente
 - dim_estado_pedido
 - dim_fecha
 - dim_pago
 - dim_producto
 - dim_vendedor**
 - fact_ventas_item
- Trigger Functions
- Types
- Views
- public
- stg
- Subscriptions
- postgres
- in/Group Roles
- namespaces
- local
- SQL 17

```
1 SELECT * FROM dw.dim_vendedor
2 ORDER BY vendedor_key ASC
```

Data Output Messages Notifications

	vendedor_key [PK] integer	seller_id character varying (32)	ciudad character varying (100)	estado character (2)
1	1	3442f8959a84dea7ee197c632cb2df15	campinas	SP
2	2	d1b65fc7debc3361ea86b5f14c68d2e2	mogi guacu	SP
3	3	ce3ad9de960102d0677a81f5d0bb7b...	rio de janeiro	RJ
4	4	c0f3eea2e14555b6faeea3dd58c1b1c3	sao paulo	SP
5	5	51a04a8a6bdc23deccc82b0b80742cf	braganca paulista	SP
6	6	c240c4061717ac1806ae6ee72be353...	rio de janeiro	RJ
7	7	e49c26c3edfa46d227d5121a6b6e4d37	brejao	PE
8	8	1b938a7ec6ac5061a66a3766e0e75f90	penapolis	SP
9	9	768a86e36ad6aae3d03ee3c6433d61df	sao paulo	SP
10	10	ccc4bb5f32a6ab2b7066a4130f114e3	curitiba	PR
11	11	8cb7c5ddf41f4d506eba76e9a4702a25	anapolis	GO
12	12	a7a9b880c49781da66651ccf4ba9ac38	itirapina	SP
13	13	8bd0f31cf0a614c658f6763bd02dea69	sao paulo	SP
14	14	05a48cc859962767935ab9087417f...	sao paulo	SP
15	15	7b8e8ec35bad4b0ef7e3963650b0a87b	tubarao	SC
16	16	144c08e64d55fb3c25f0f09c07ffcf2	lauro de freitas	BA
17	17	166e8f1381e09651983c38b1f6f91c11	imbituba	SC
18	18	e38db885400cd35c71df162f2c1dbcf	brasilia	DF

- FTS Templates
- Foreign Tables
- Functions
- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (7)
 - dim_cliente
 - dim_estado_pedido
 - dim_fecha
 - dim_pago
 - dim_producto
 - dim_vendedor
 - fact_ventas_item**
- Trigger Functions
- Types
- Views
- public
- stg
- Subscriptions
- postgres
- in/Group Roles
- namespaces
- local

```
1 SELECT * FROM dw.fact_ventas_item
2 ORDER BY order_id ASC, order_item_id ASC
```

Data Output Messages Notifications

	order_id [PK] character varying (32)	order_item_id [PK] smallint	fecha_compra_key integer	fecha_entrega_key integer	cliente_key integer	producto_key integer	vendedor_key integer	estado_key smallint	pago_key smallint	price numeric
1	00010242f6c5a6d1ba2dd792cb162...	1	20170913	20170920	50586	25866	514	1	2	
2	00018f77f20320c557190d7a144bd63	1	20170426	20170512	88075	27231	472	1	2	
3	000229ec398224ef6ca0657da4fc703e	1	20180114	20180122	21068	22625	1825	1	3	
4	00024acbcdf0a6daa1e931b038114c...	1	20180808	20180814	65816	15404	2024	1	2	
5	00042b26cf9d7ce9d9fabb4e55b4f9f	1	20170204	20170301	37776	8863	1598	1	2	
6	00048cc3ae777c65dbb7d2a0634bc1...	1	20170515	20170522	50121	3940	660	1	6	
7	00054e8431b9d7675808cb819fb4a...	1	20171210	20171218	37295	22293	2974	1	1	
8	000576e39319847cbb9d288c5617fa6	1	20180704	20180709	94882	6975	692	1	4	
9	0005a1a1728c9d785b8e2b08b90457...	1	20180319	20180329	37389	2714	68	1	2	
10	0005f50442cb953dcd1d21e1fb923495	1	20180702	20180704	2833	28256	1158	1	1	
11	00061f2a7b0c09da83e415a52dc8a4f1	1	20180324	20180329	6226	14395	1414	1	2	
12	00063b381e2406e52a429470734eb...	1	20180727	20180807	23934	23294	849	1	3	
13	0006ec9db01a64e59a68b2c340bf65...	1	20180724	20180731	47565	9662	2464	1	3	
14	0008288aa423d2a3f00cb17cd7d8719	1	20180213	20180226	59268	4599	475	1	6	
15	0008288aa423d2a3f00cb17cd7d8719	2	20180213	20180226	59268	4599	475	1	6	
16	0009792311464db532f7765bf7b182ae	1	20180814	20180822	27577	1132	2122	1	6	
17	0009c9a177916a706d71784483a5d6...	1	20180425	20180430	36208	9419	1522	1	3	

Query Query History

```
1 SELECT version();
2 SELECT dw.fn_total_ventas();
```

Data Output Messages Notifications

	fn_total_ventas numeric
1	13449529.68

Query Query History

```

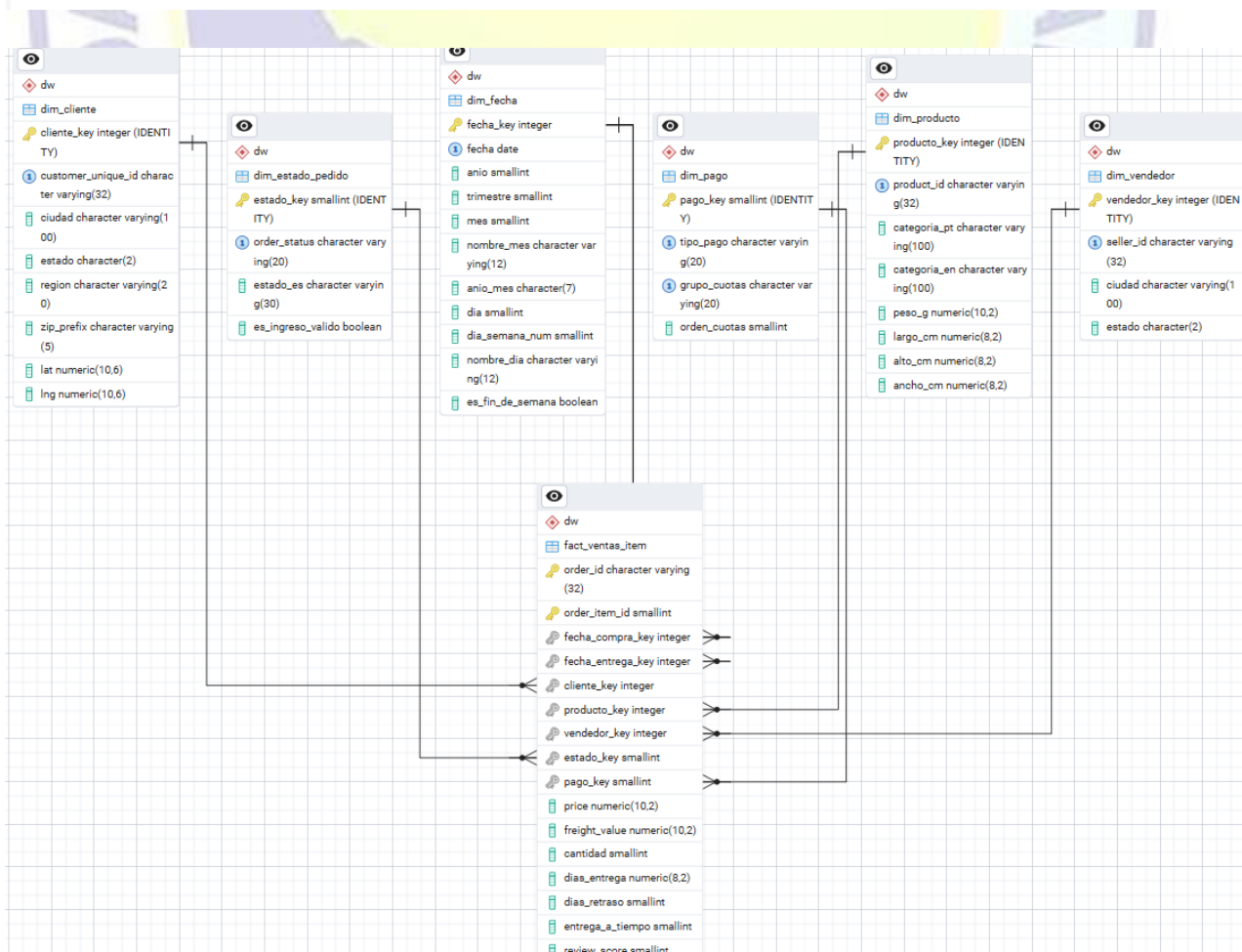
1  SELECT 'fact_ventas_item' AS tabla, COUNT(*) FROM dw.fact_ventas_item
2  UNION ALL SELECT 'dim_cliente', COUNT(*) FROM dw.dim_cliente
3  UNION ALL SELECT 'dim_producto', COUNT(*) FROM dw.dim_producto
4  UNION ALL SELECT 'dim_vendedor', COUNT(*) FROM dw.dim_vendedor
5  UNION ALL SELECT 'dim_fecha', COUNT(*) FROM dw.dim_fecha
6  UNION ALL SELECT 'dim_estado_pedido', COUNT(*) FROM dw.dim_estado_pedido
7  UNION ALL SELECT 'dim_pago', COUNT(*) FROM dw.dim_pago;

```

Data Output Messages Notifications



	tabla text	count bigint
1	dim_estado_pedido	8
2	dim_pago	25
3	dim_fecha	1461
4	dim_vendedor	3095
5	dim_producto	32951
6	fact_ventas_item	112279
7	dim_cliente	95774



10. Preguntas para defender el trabajo

- **¿Qué representa una fila de tu tabla de hechos y por qué elegiste esa granularidad?**

Una fila de `fact_ventas_item` es un ítem (producto) vendido dentro de un pedido, identificado por (`order_id`, `order_item_id`). Elegí el ítem porque es el nivel más detallado del dataset. Desde ahí puedo agrupar hacia arriba por pedido, producto, categoría, vendedor, cliente o mes. Si hubiera elegido el pedido, no podría analizar categorías ni vendedores, porque un pedido puede mezclar varios productos y varios vendedores.

- **¿Por qué existe la zona de staging y qué pasaría si cargaras directo a las dimensiones?**

El staging (esquema `stg`) guarda los datos ya limpios en tablas simples. Desde ahí construyo dimensiones y hechos con SQL. Separar las etapas me permite repetir la carga del modelo sin volver a limpiar los CSV, y comparar origen y destino: validé que ítems en staging y filas en hechos fueran iguales (diferencia 0). Si cargara directo, mezclaría limpieza, claves y reglas del modelo en un solo paso, y sería difícil encontrar dónde se pierde o se duplica un dato.

- **¿Para qué sirve cada clave foránea y qué ocurre si intentas insertar un hecho con un `cliente_key` que no existe?**

Cada clave foránea conecta la tabla de hechos con una dimensión (fecha de compra, fecha de entrega, cliente, producto, vendedor, estado y pago). Garantiza que ningún hecho quede huérfano. Si inserto un hecho con un `cliente_key` inexistente, PostgreSQL rechaza la fila con el error `violates foreign key constraint`. Por eso mi validación de integridad da 0 en todos los `sin_*`.

- **¿Por qué `customer_id` no sirve para medir recompra y qué usaste en su lugar?**

En Olist, `customer_id` se genera de nuevo en cada pedido. La misma persona aparece con varios `customer_id` distintos, así que nadie parecería recomprar. Usé `customer_unique_id`, que identifica a la persona real. Por eso `dim_cliente` tiene 95,774 clientes únicos frente a 99,092 `customer_id` originales. La tasa de recompra es clientes con más de un pedido dividido entre clientes únicos (3.03 %).

- **¿Por qué no se puede promediar `review_score` directamente sobre la tabla de hechos? ¿Cómo lo resolviste?**

La reseña es de un pedido, pero el modelo está a nivel de ítem, así que el mismo valor se repite en cada ítem del pedido. Un pedido con 3 ítems pesaría 3 veces. Lo resolví con la función `dw.fn_base_pedidos`, que primero agrupa por `order_id` y deja una fila por pedido. Los KPI de calificación, entrega y ticket promedio se calculan sobre esa base. La calificación promedio es 4.12.

• **¿Qué diferencia hay entre una vista y una función? ¿Cuándo usaste cada una?**

Una vista es una consulta guardada, sin parámetros, que siempre devuelve lo mismo. Una función recibe parámetros (fechas, categoría, estado) y devuelve un KPI o una tabla filtrada. Usé **vistas** para componentes fijos del dashboard, como `v_ventas_estado`, `v_metodos_pago` y `v_detalle_ventas`. Usé **funciones** para lo que depende de los filtros, como `fn_total_ventas()`, `fn_ticket_promedio()` y `fn_top_categorias()`. Un parámetro en NULL significa "sin filtro".

• **¿Por qué fecha_entrega_key puede ser NULL y cómo lo validaste?**

Porque un pedido que aún no se entrega (enviado, procesando, cancelado) no tiene fecha de entrega, y inventar una fecha falsearía los tiempos. Por eso la columna admite NULL, y la FK solo aplica cuando hay valor. Lo validé con consultas que cuentan los ítems sin fecha de entrega y los agrupan por estado del pedido. Se concentran en pedidos no entregados. Unos pocos pedidos "delivered" no tienen fecha desde el dataset original, y los conservé a propósito.

• **¿Qué hace docker compose down y en qué se diferencia de down -v?**

docker compose down apaga y elimina los contenedores y la red, pero **conserva el volumen** postgres_data, donde viven los datos. Al volver a encender con docker compose up -d db, la base sigue con todo cargado. docker compose down -v hace lo mismo y además **borra el volumen**, o sea, borra la base de datos. Para reconstruir todo después hay que ejecutar docker compose run --rm etl. Uso down para pausar y down -v para empezar de cero.

11. Errores comunes y cómo resolverlos

Síntoma	Causa probable	Solución
Cannot connect to the Docker daemon	Docker Desktop no está abierto	Abre Docker Desktop, espera a que diga "running" y repite
port is already allocated (5432)	Ya hay un PostgreSQL local usando ese puerto	Detén ese servicio, o cambia en el compose "5433:5432" y usa el puerto 5433
Error de conexión en el ETL	Contenedor apagado o aún iniciando	docker compose up -d, espera 10–20 s, revisa con docker logs bi-postgres
faltan archivos en data/raw	CSV mal ubicados	Descomprime los 9 CSV directamente en data/raw/
password authentication failed	Se cambió la contraseña después de crear el volumen	docker compose down -v y volver a levantar (borra datos)
relation ... does not exist	Se ejecutó un script fuera de orden	Ejecuta en orden: ETL → 02 → 03 → 04 → 05
violates foreign key constraint	Se cargó hechos sin dimensiones	Re-ejecuta 03 completo (vacía y carga en orden)
ModuleNotFoundError: pandas / psycopg2	Falta instalar librerías	pip install pandas psycopg2-binary

Pregunta de la actividad: ¿Cómo voy a almacenar, consultar y presentar la información?

Almacenar: limpio los 9 CSV de Olist con Python y los cargo a un esquema de staging en PostgreSQL, que corre en un contenedor Docker. De ahí construyo un modelo estrella con una tabla de hechos a nivel de ítem y 6 dimensiones (fecha, cliente, producto, vendedor, estado y pago), con claves primarias y foráneas.

Consultar: valido la carga con consultas SQL (conteos, integridad, nulos y totales). La lógica de negocio queda en vistas y funciones SQL, que reciben filtros y devuelven los KPI ya calculados.

Presentar: un dashboard diseñado en 3 mockups, con KPI, filtros, gráficos y una tabla de detalle. Cada componente se conecta a un objeto SQL mediante la matriz de trazabilidad. Por ejemplo, la tarjeta de ingresos usa `fn_total_ventas()`. Todo el proyecto está en un repositorio de GitHub y se reproduce con `docker compose run --rm etl`.

